**Lab -9:** **Experiment No: 9**

## 1. Aim

To develop an ERC 20 token using Solidity in Remix IDE and deploy it on an Ethereum testnet using MetaMask. To understand token attributes, transfers, allowances, and events.

## 2. Theory

ERC 20 is a standard for creating fungible tokens on the Ethereum blockchain. Fungible tokens are interchangeable, meaning each token has equal value and characteristics.

In this experiment, a Solidity smart contract named MyToken is created with token properties such as name, symbol, decimals, and totalSupply. The balanceOf mapping stores token balances, while the allowance mapping maintains spending permissions.

The contract contains three main functions:

1. transfer(): Sends tokens from one account to another.
2. approve(): Grants permission to another account to spend tokens.
3. transferFrom(): Transfers tokens using an approved allowance.

The Transfer and Approval events record token transactions and permission changes. The constructor generates the initial token supply and assigns it to the deploying account.

Remix IDE is used to write, compile, and deploy the contract, while MetaMask connects to the Ethereum testnet and approves transactions.

ERC 20 tokens are fungible, whereas ERC 721 tokens represent unique, non-fungible assets such as NFTs.

## 3. Procedure

1. Open Remix IDE and explore the LearnETH ERC 20 Token tutorial.
2. Study the six chapters explaining ERC 20 token functionality.
3. Install MetaMask and configure an Ethereum testnet.
4. Obtain test ETH from a faucet for deployment.
5. Create a Solidity file named MyToken.sol in Remix IDE.
6. Write the ERC 20 token contract with the required attributes and functions.
7. Compile the contract using Solidity compiler version 0.8.20 or compatible.
8. Select Browser Extension in Deploy & Run Transactions and connect MetaMask.
9. Enter the initial token supply and deploy the contract.
10. Confirm the deployment transaction in MetaMask.

11. Expand the deployed contract and verify token attributes, balances, and total supply.

**4. Code**

File Name: MyToken.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract MyToken {
    string public name = "Khushi Token";
    string public symbol = "KHS";
    uint8 public decimals = 18;
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;
    event Transfer(
        address indexed from,
        address indexed to,
        uint256 value
    );
    event Approval(
        address indexed owner,
        address indexed spender,
        uint256 value
    );
    constructor(uint256 initialSupply) {
        totalSupply = initialSupply * 10 ** decimals;
        balanceOf[msg.sender] = totalSupply;
        emit Transfer(address(0), msg.sender, totalSupply);
    }
    function transfer(
        address to,
        uint256 amount
    ) public returns (bool) {
        require(
            balanceOf[msg.sender] >= amount,
            "Insufficient balance"
        );
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;
        emit Transfer(msg.sender, to, amount);
        return true;
```

```
    }
    function approve(
        address spender,
        uint256 amount
    ) public returns (bool) {
        allowance[msg.sender][spender] = amount;
        emit Approval(msg.sender, spender, amount);
        return true;
    }
    function transferFrom(
        address from,
        address to,
        uint256 amount
    ) public returns (bool) {
        require(
            balanceOf[from] >= amount,
            "Insufficient balance"
        );
        require(
            allowance[from][msg.sender] >= amount,
            "Allowance exceeded"
        );
        balanceOf[from] -= amount;
        balanceOf[to] += amount;
        allowance[from][msg.sender] -= amount;
        emit Transfer(from, to, amount);
        return true;
    }
}
```

**5. Output**

**Fig. 1.** MetaMask transaction request for deploying MyToken

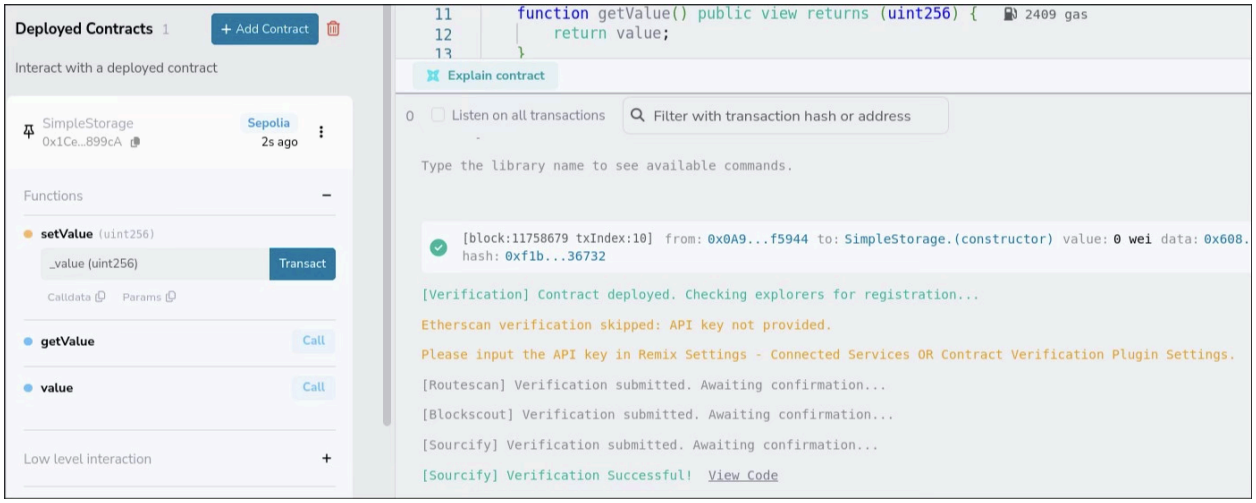



**Fig. 2.** MyToken deployment request and Remix deployment panel

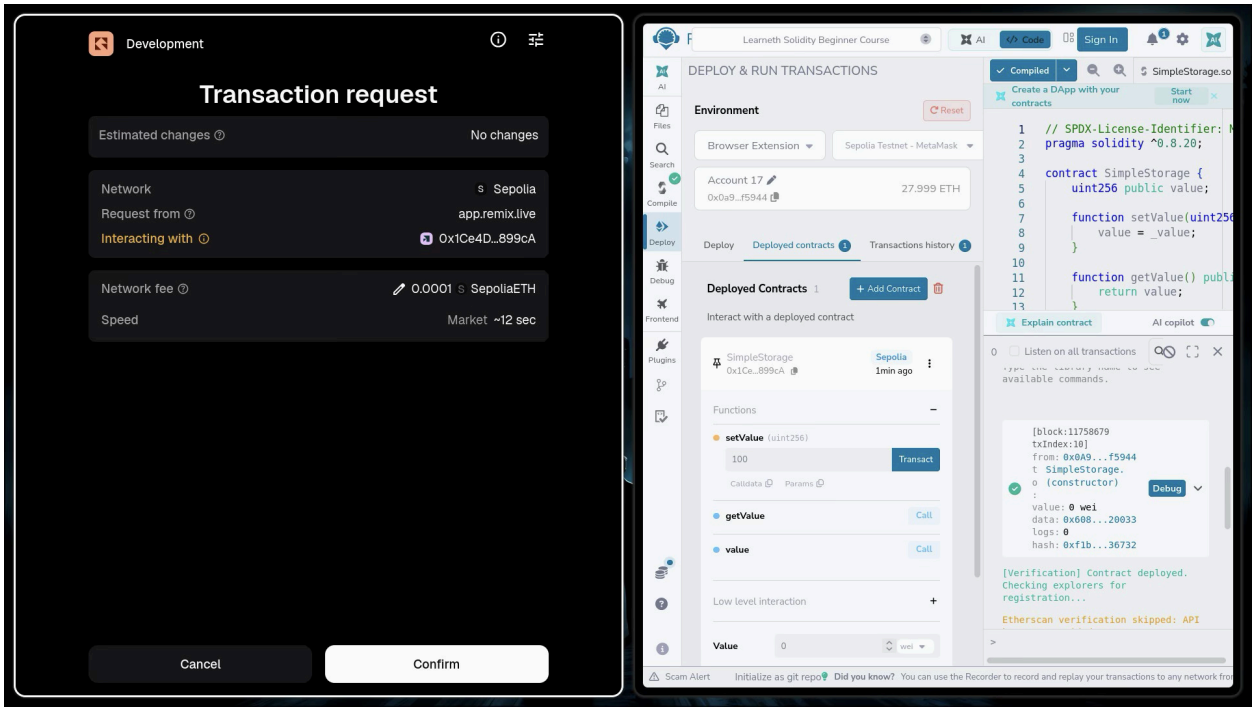


**Fig. 3.** Deployed MyToken contract with transaction details

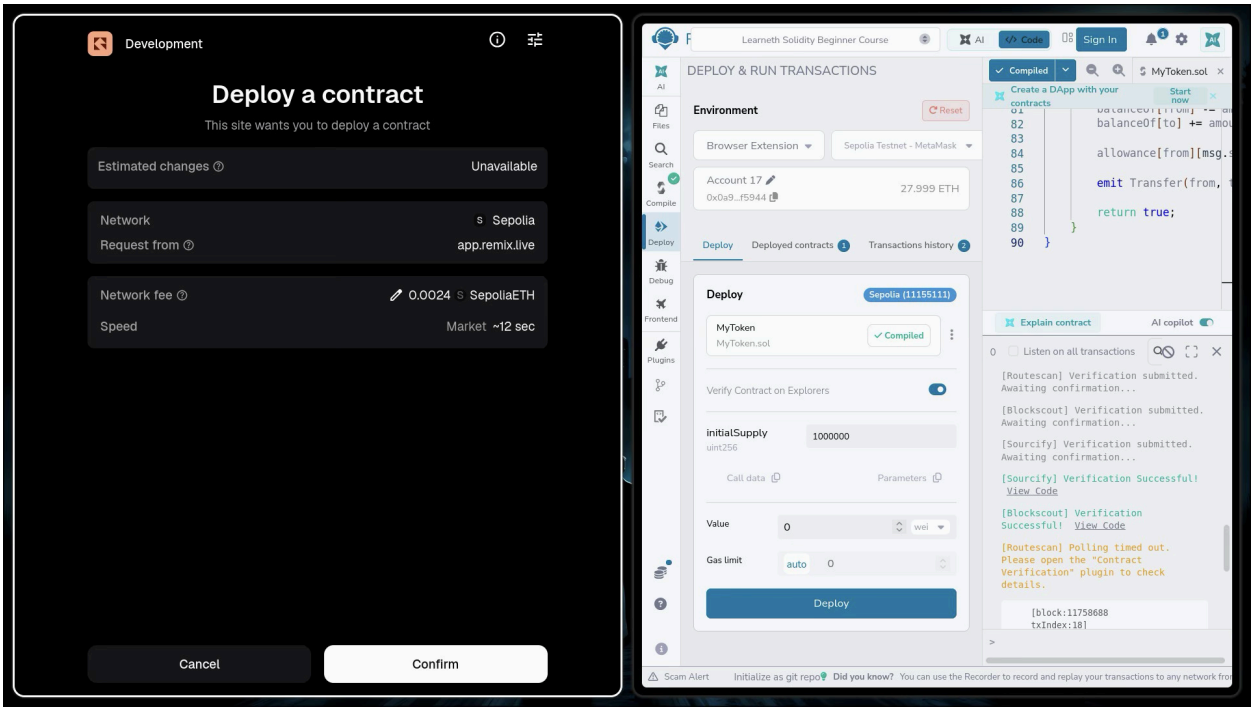



**Fig. 4.** MyToken compiler panel in Remix IDE

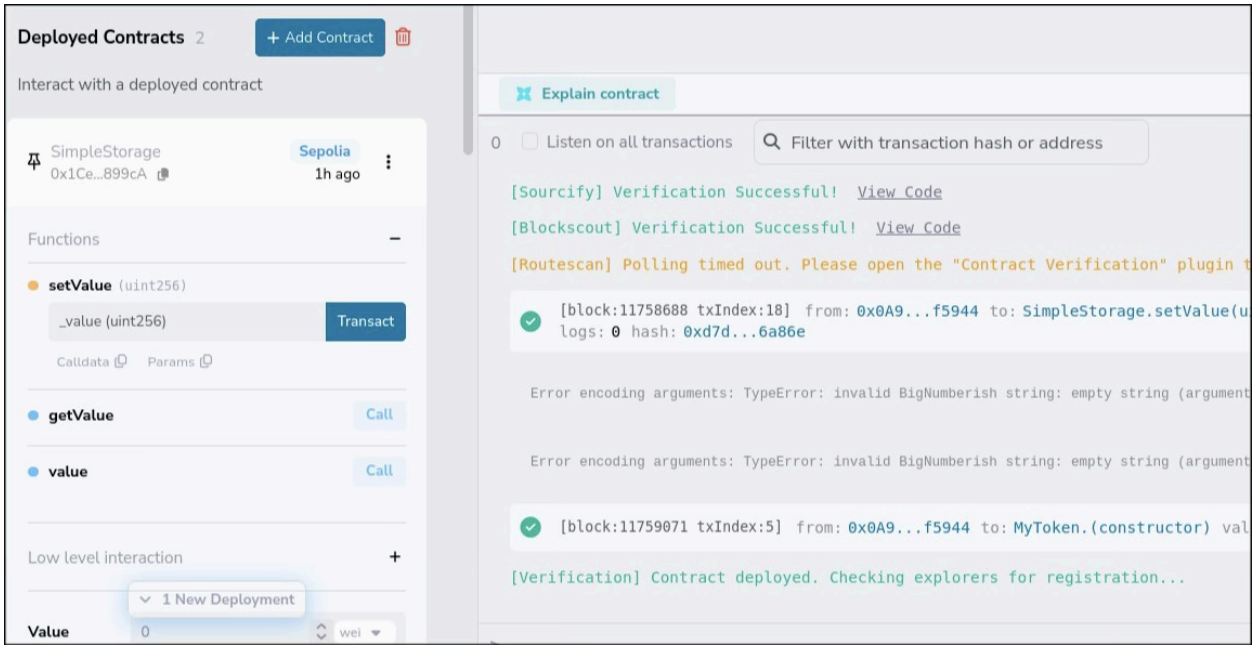


**Fig. 5.** MyToken public attributes and call results

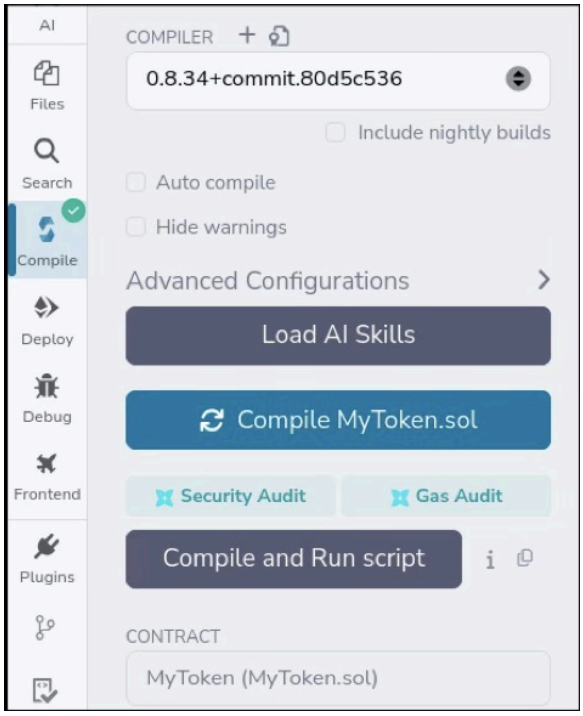
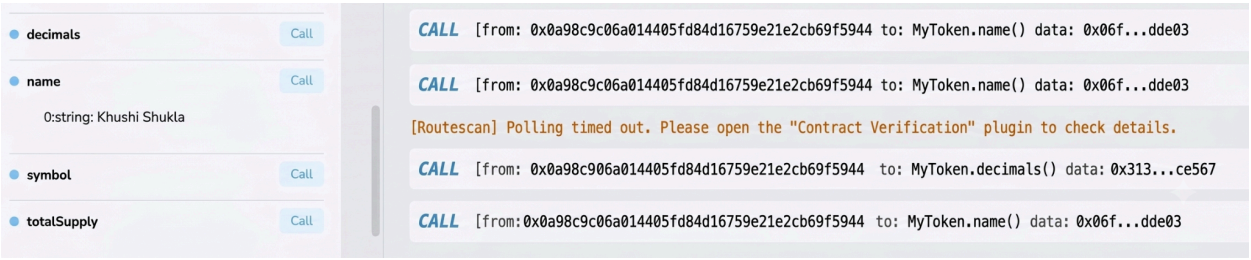



**Fig. 6.** MyToken total supply call result




## 6. Conclusion

The MyToken smart contract demonstrates the creation of an ERC 20-style fungible token using Solidity. The experiment covers token attributes, balance management, transfer functions, allowances, and events. It also demonstrates how Remix IDE and MetaMask can be used to compile and deploy smart contracts on an Ethereum testnet.

Note: This is a simplified ERC 20 implementation. It demonstrates the main token functions but is not fully ERC 20-compliant because it omits certain standard interface requirements and zero-address checks.